

软件详细设计说明书

属性	内容
系统名称	松果集市
编制单位	软件工程基础第 3 组
修订日期	2026-08-28

修订记录

版本	日期	修订说明
V1.0	2026-06-24	初版文档。
V2.2	2026-08-28	同步控制器、服务、数据访问、购物车与地址数据结构、部署方式和测试矩阵。
V2.3	2026-08-29	统一 UC-01 ~ UC-14，按现有图片核对对象级动态模型覆盖，并区分订单目标模型与当前代码行为。

目录

软件详细设计说明书

修订记录

目录

1 文档说明

- 1.1 编写目的
- 1.2 设计范围
- 1.3 参考资料
- 1.4 图形约定

2 总体设计

- 2.1 总体架构
- 2.2 系统模块划分
- 2.3 组件及接口关系
- 2.4 前后端调用链路

3 用例设计

- 3.1 总体用例
- 3.2 社区互动用例
- 3.3 统一用例与实现模块对应

4 静态结构设计

- 4.1 概念类设计
- 4.2 后端分层类职责
- 4.3 控制器设计
- 4.4 服务与数据访问设计
- 4.5 认证与用户模块方法设计
- 4.6 商品、订单与社区模块方法设计

- 4.7 聊天模块方法设计
 - 5 关键模块动态设计
 - 5.1 UC-01 注册并登录
 - 5.2 UC-06 卖家发布商品与 UC-07 管理员审核商品
 - 5.3 UC-08 创建订单与 UC-09 查询评价
 - 5.4 UC-11 建立会话、收发消息并使用 AI 议价
 - 5.5 UC-06、UC-07 商品状态设计
 - 5.6 UC-08、UC-09 订单目标状态设计
 - 5.7 UC-14 用户账号状态设计
 - 5.8 UC-10 创建话题并完成社区互动
 - 5.9 UC-12 提交实名认证并由管理员审核
 - 5.10 现有对象级动态模型覆盖
 - 6 数据设计
 - 6.1 用户与店铺数据
 - 6.2 商品、订单与评价数据
 - 6.3 社区与聊天数据
 - 6.4 数据约束与索引
 - 6.5 核心表字段设计
 - 6.5.1 users 用户表
 - 6.5.2 goods 商品表
 - 6.5.3 orders 与 product_review
 - 6.5.4 conversation 与 chat_message
 - 6.5.5 社区与互动表
 - 6.5.6 cart_item 与 user_address
 - 6.6 数据一致性与事务边界
 - 7 接口与公共机制设计
 - 7.1 接口规范
 - 7.2 主要 REST 接口
 - 7.3 WebSocket 设计
 - 7.4 认证、异常与上传
 - 7.5 关键接口输入输出设计
 - 7.5.1 登录接口
 - 7.5.2 发布商品接口
 - 7.5.3 创建订单接口
 - 7.5.4 创建会话接口
 - 7.5.5 商品审核接口
 - 7.6 统一响应与错误传播
 - 8 部署、测试与实现边界
 - 8.1 部署架构
 - 8.2 容器运行设计
 - 8.3 测试重点
 - 8.4 实现约束与版本边界
 - 8.5 部署配置项设计
 - 8.6 模块测试矩阵
-

1 文档说明

1.1 编写目的

本文档说明松果集市的总体结构、业务模块、类与组件关系、关键对象状态、业务交互过程、数据关系和部署方式。文档以 UML 模型和数据关系图为主要表达方式，为编码、联调、测试、部署和后续维护提供详细设计依据。

1.2 设计范围

设计范围包括账号与认证、商品与店铺、服务端购物车与地址、订单与评价、社区互动、聊天与 AI 议价、平台治理、文件上传、MySQL 持久化、WebSocket 通信、Docker Compose 与 Kubernetes 部署。

本文档不使用运行界面截图。界面操作说明由《用户手册》负责，本文档只描述界面动作背后的软件结构和处理过程。

1.3 参考资料

1. 教师示例《软件设计说明书》中的软件详细设计部分。
2. 松果集市《软件需求规格说明书》和《软件概要设计说明书》。
3. 项目组在 CodeArts 中绘制的用例图、组件图、概念类图、顺序图、状态图和活动图。
4. `shopping_front`、`shopping_back` 和 `deploy` 目录中的当前代码与配置。

1.4 图形约定

图形	用途
用例图	描述参与者与系统功能边界
组件图	描述前端、接口和后端业务组件的依赖关系
类图	描述核心领域类、属性、方法、关系和多重性
顺序图	描述一次业务请求中对象之间的消息顺序
活动图	描述包含判断与分支的业务执行过程
状态图	描述商品、订单等对象的生命周期
ER/关系图	描述数据库表及其关联字段

2 总体设计

2.1 总体架构

松果集市交易平台总体架构图



图 2-1 松果集市总体架构图

系统从上到下划分为用户访问层、前端应用层、接入层、认证与控制层、业务服务层以及数据与资源层。H5 和小程序共享 uni-app/Vue 前端代码；REST API 承担普通业务调用，WebSocket 承担实时聊天；Spring Boot 内部按业务域组织服务；MySQL 保存核心业务数据，上传目录保存图片，外部 AI API 提供议价、问答和发布建议。

图中的 Redis 表示概要设计阶段预留的缓存扩展点。当前代码与 Docker Compose 没有启用 Redis，详细实现仍以 MySQL、内存 Token 和本地上传目录为准。

2.2 系统模块划分

松果集市系统模块结构图

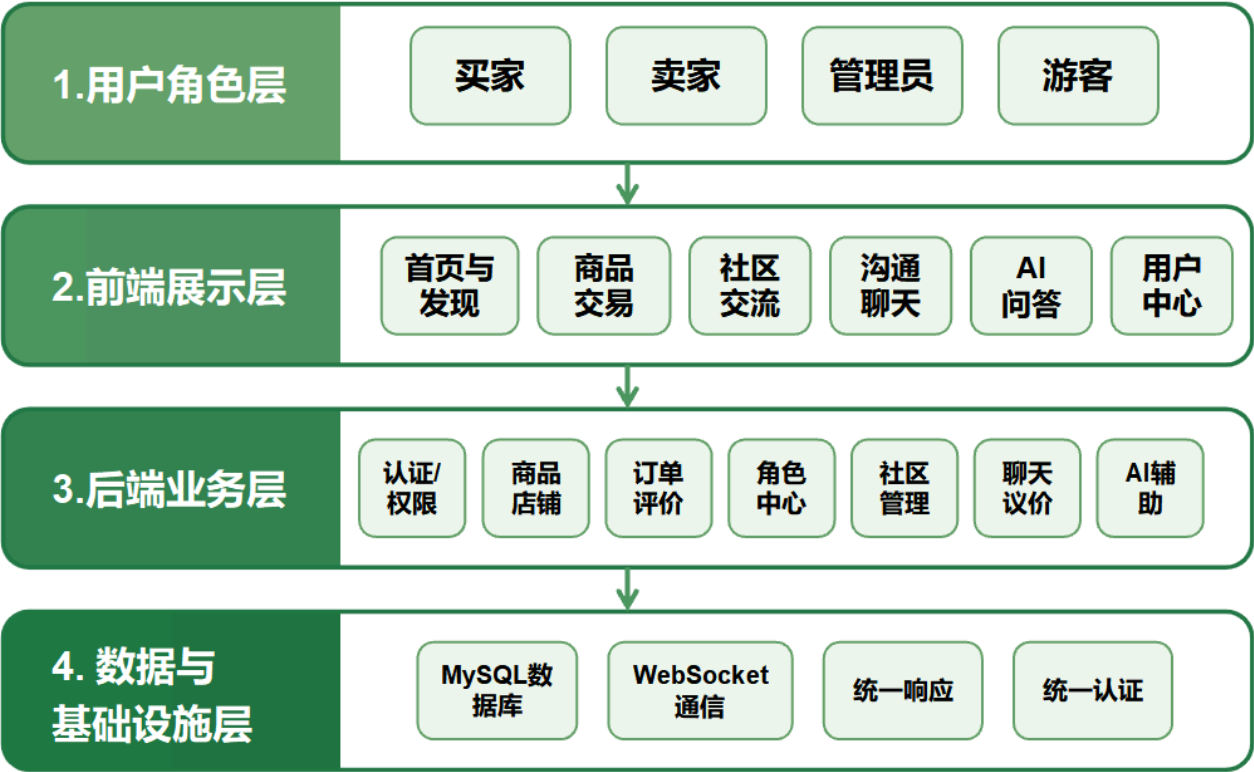


图 2-2 松果集市系统模块结构图

用户角色层包含访客、买家、卖家和管理员；前端展示层承接首页发现、商品交易、社区、聊天、AI 和用户中心；后端业务层划分为认证权限、商品店铺、订单评价、角色中心、社区管理、聊天议价和 AI 辅助；数据与基础设施层提供 MySQL、WebSocket、统一响应和统一认证。

- 1. 前端模块不直接访问数据库。
- 2. Controller 只负责协议适配、参数装配和响应包装。
- 3. Service 负责权限、业务校验、状态变更和数据组合。
- 4. Mapper 负责 SQL 执行和字段映射。
- 5. AI 模块属于可降级扩展，故障时不得影响普通交易和聊天。

2.3 组件及接口关系

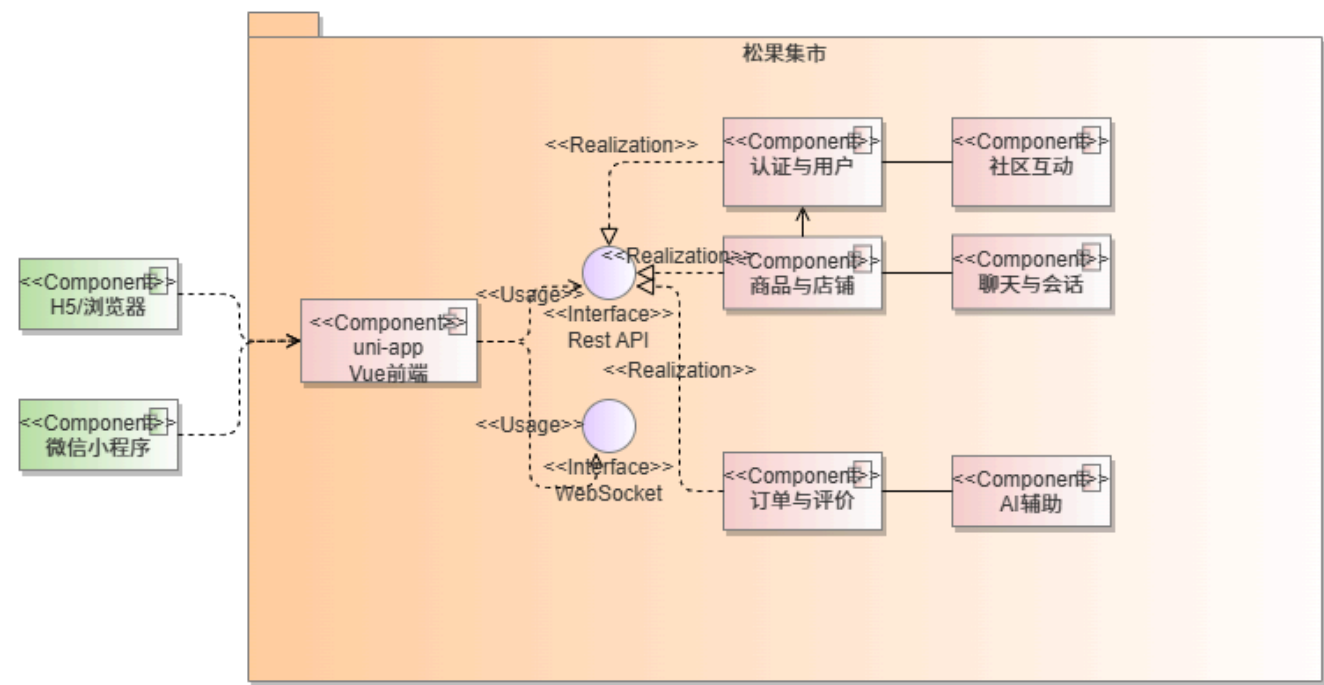


图 2-3 松果集市组件图

H5 浏览器和微信小程序使用 uni-app/Vue 前端组件。前端通过 REST API 使用认证用户、商品店铺、订单评价、社区互动和 AI 辅助组件，通过 WebSocket 使用聊天会话组件。

组件	提供能力	依赖
认证与用户	注册、登录、当前用户、角色和资料	REST API、用户数据
商品与店铺	商品查询、发布、审核、店铺和关注	REST API、商品与店铺数据
订单与评价	订单创建、查询和评价	REST API、订单数据
社区互动	话题、帖子、评论、点赞和动作	REST API、社区数据
聊天与会话	会话、消息、已读和转人工	REST API、WebSocket、会话数据
AI 辅助	AI 议价、智能问答、发布建议	REST API、外部 AI API

2.4 前后端调用链路

前后端请求调用流程图



图 2-4 前后端请求调用流程图

普通业务请求依次经过页面、前端 services、统一 request、Controller、Service、Mapper 和数据库。返回结果按相反方向传递，并由 ApiResult 统一封装。实时消息在完成身份认证后通过 WebSocket 通道处理，但消息保存仍经过聊天 Service 和 Mapper。

3 用例设计

3.1 总体用例

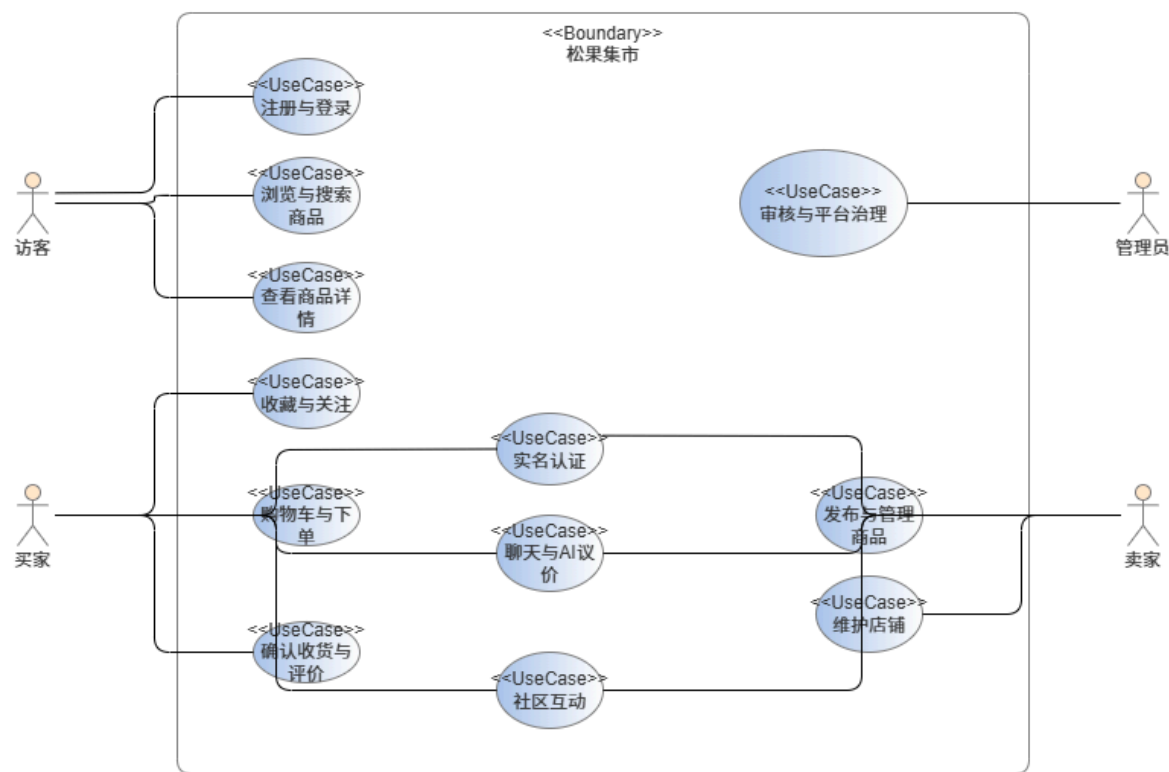


图 3-1 松果集市总体用例图

- 访客可注册登录、浏览搜索商品和查看商品详情。
- 买家可收藏关注、使用购物车下单、确认收货评价、实名认证、聊天议价和参与社区。
- 卖家可发布管理商品、维护店铺、接受聊天议价并参与社区。
- 管理员负责审核商品、实名认证、用户与平台治理。

参与者与用例之间使用 Association；必须复用的公共行为可使用 Include；条件性附加行为可使用 Extend。业务图中不使用普通 Dependency 代替用例关系。

3.2 社区互动用例

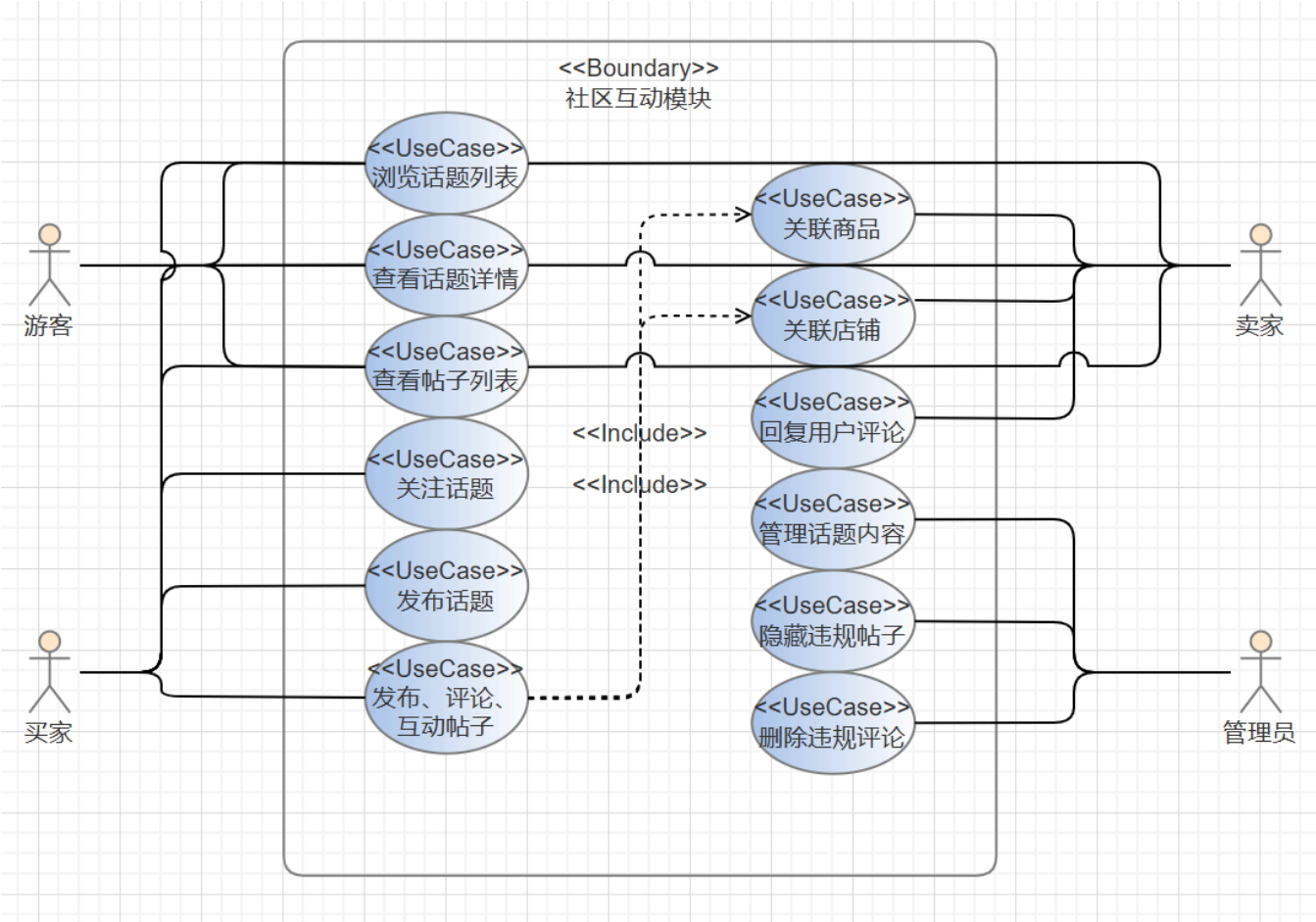


图 3-2 社区互动模块用例图

访客可以浏览话题、查看详情和帖子列表；登录买家或卖家可以关注话题、发布话题、发布帖子、评论和互动；卖家可以将商品或店铺关联到帖子；管理员承担违规内容治理。浏览属于公开读操作，写入和互动操作均需登录。

3.3 统一用例与实现模块对应

以下编号与 00-统一用例清单.md 完全一致。

用例	Controller	Service	主要数据/外部依赖
UC-01 注册并登录	AuthController	AuthService	users、内存 Token
UC-02 浏览、搜索并查看商品	ShopController	ShopService	goods、store
UC-03 收藏商品、关注店铺并查看	CenterController、ShopController	CenterService、ShopService	favorite_goods、follow_store

用例	Controller	Service	主要数据/外部依赖
UC-04 维护个人资料与头像	AuthController、 UploadController	AuthService、上传处理	users、上传文件卷
UC-05 卖家查看并维护店铺	ShopController	ShopService	store
UC-06 卖家发布并维护商品	ShopController、 UploadController	ShopService	goods、上传文件卷、可选 AI
UC-07 管理员审核商品	ShopController	ShopService	goods
UC-08 维护购物车和地址并创建订单	CartController、 AddressController、 ShopController	CartService、 AddressService、 ShopService	cart_item、 user_address、orders； 当前页面仍使用本地购物车/地址
UC-09 查询订单并提交评价	ShopController	ShopService	orders、 product_review、 credit_record
UC-10 创建话题并完成社区互动	ShopController	ShopService	话题、帖子、评论和动作表
UC-11 建立会话、收发消息并使用 AI 议价	ChatController、 ChatWebSocketController	会话、消息、AI 服务	conversation、 chat_message、外部 AI
UC-12 提交实名认证并由管理员审核	CenterController	CenterService	user_realname_auth
UC-13 使用 AI 购物助手	AiAssistantController	AiAssistantService	外部 AI/规则降级
UC-14 管理员维护用户状态与信用	CenterController、 AuthController	CenterService、 AuthService	users、credit_record

4 静态结构设计

4.1 概念类设计

松果集市概念类图

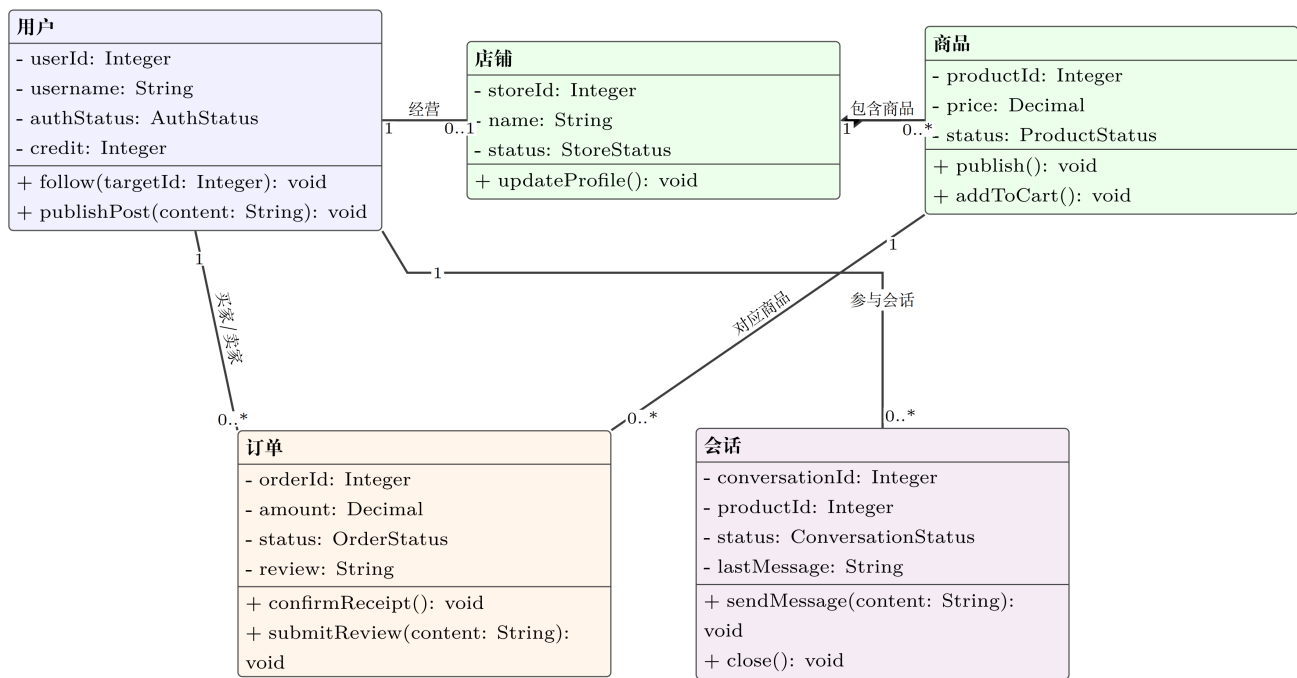


图 4-1 松果集市概念类图

概念类图保留五个核心领域类，避免把属性或操作误画成独立 Class。

类	核心属性	主要操作	关系
用户	userId: Integer、username: String、authStatus: AuthStatus、credit: Integer	关注、发布帖子	可经营店铺，并产生订单和会话
店铺	storeId: Integer、name: String、status: StoreStatus	更新店铺资料	组合多个商品
商品	productId: Integer、price: Decimal、status: ProductStatus	发布、加入购物车	可对应多个历史订单或会话
订单	orderId: Integer、amount: Decimal、status: OrderStatus、review: String	确认收货、提交评价	对应买家、卖家和商品
会话	conversationId: Integer、productId: Integer、status: ConversationStatus、lastMessage: String	发送消息、关闭	对应买家、卖家和商品

图中的方法表示领域行为，不要求在持久化类中存在同名方法。当前代码将多数行为放在 Service 中，把 Record/Model 作为数据载体。

4.2 后端分层类职责

层次	类/接口	职责
Controller	AuthController、CenterController、ShopController、CartController、AddressController、ChatController、UploadController	接收请求和认证头，调用服务并返回统一结果
WebSocket Controller	ChatWebSocketController	接收 STOMP 消息并推送实时消息与回执
Service	AuthService、CenterService、ShopService、CartService、AddressService	业务校验、角色权限、状态处理和视图组装
Chat Service	ConversationService、ChatMessageService、AiBargainService、AiAssistantService	会话、消息、AI 议价与降级
Mapper	UserMapper、CenterMapper、Shop*Mapper、CartMapper、AddressMapper、ConversationMapper、ChatMessageMapper	MyBatis SQL 与字段映射
Model/DTO	用户、商品、订单、会话模型及请求/响应 DTO	层间数据传递

4.3 控制器设计

控制器	主要接口组	设计要求
AuthController	/api/auth/**	注册登录校验、Bearer Token 提取
CenterController	/api/center/**	角色中心、实名、信用和用户治理
ShopController	商品、店铺、订单、话题和审核接口	统一调用 ShopService
CartController	/api/cart/**	本人购物车的列表、添加、改量、勾选和删除
AddressController	/api/address/**	本人地址的列表、新增、修改、删除和默认地址切换
ChatController	/api/chat/**	会话校验、消息、AI 议价和转人工
ChatWebSocketController	/app/**	实时消息和已读回执
UploadController	/api/upload/image	文件校验、保存并返回访问地址
AiAssistantController	/api/ai/assistant	通用 AI 助手入口

Controller 不直接访问 Mapper，也不只依靠前端按钮控制权限。

4.4 服务与数据访问设计

服务	关键职责	主要 Mapper
AuthService	BCrypt、Token、账号状态、用户资料	UserMapper
CenterService	角色中心、实名认证、信用和治理	CenterMapper
ShopService	商品店铺、订单评价、社区互动	ShopProductMapper、ShopStoreMapper、ShopOrderMapper、ShopTopicMapper
CartService	商品存在性、自购限制、数量及购物车归属校验	CartMapper、ShopProductMapper
AddressService	地址必填项、地址归属、首个默认地址和默认地址切换	AddressMapper
ConversationService	创建/复用会话、参与者和状态	ConversationMapper
ChatMessageService	消息、历史、未读和已读	ChatMessageMapper
AI 服务	议价、助手和失败降级	外部 AI API

Mapper 使用 MyBatis 注解 SQL；Service 负责组合多个 Mapper 的结果，Mapper 不包含跨业务域流程判断。

4.5 认证与用户模块方法设计

认证模块由 `AuthController`、`AuthService`、`UserMapper`、认证请求 DTO 和用户视图组成。Controller 只负责 HTTP 映射与 Bearer Token 提取；密码校验、账号状态、角色规范化和令牌签发均位于 Service。

所属类	方法	输入	输出	处理要点
AuthController	register(body)	RegisterRequest	ApiResponse<LoginResponse>	使用 @Valid 校验后调用服务
AuthController	login(body)	LoginRequest	ApiResponse<LoginResponse>	登录成功同时返回 Token 与用户视图
AuthController	me(authorization)	Bearer Token	ApiResponse<AuthUserView>	统一提取 Token，不接受错误前缀
AuthController	updateProfile(authorization,body)	Token、头像地址	AuthUserView	只允许安全的文件或 HTTP(S) 地址
AuthService	register(req)	用户名、密码、手机号、角色	LoginResponse	用户名唯一；BCrypt；禁止公开注册管理员
AuthService	login(req)	用户名、密码	LoginResponse	比对散列；拒绝禁用账号；签发随机 Token
AuthService	me(token)	Token	AuthUserView	Token 不存在、过期或用户不存在时返回 401
UserMapper	findByUsername(username)	用户名	StoredUser	登录和唯一性校验共用
UserMapper	insertUser(user)	StoredUser	影响行数	仅写入密码散列，不写明文密码

`StoredUser` 是持久化模型，字段包括 `userId`、`username`、`passwordHash`、`phone`、`credit`、`role`、`status` 和 `avatarUrl`。`AuthUserView` 是对外视图，不包含 `passwordHash`，手机号以脱敏形式返回。

4.6 商品、订单与社区模块方法设计

`ShopService` 是模块化单体中的业务协调服务。为防止类职责失控，其公共方法按商品店铺、订单评价和社区三组理解，并分别委托不同 Mapper。

方法组	代表方法	核心校验	数据访问
商品读取	<code>products(scene,keyword)</code> 、 <code>product(id)</code> 、 <code>myProducts(user)</code>	场景和关键词规范化；公开列表只返回已通过商品	<code>ShopProductMapper</code>
商品发布审核	<code>publish(request,user)</code> 、 <code>pendingProducts()</code> 、 <code>audit(id,request)</code>	卖家/管理员角色、价格、状态、拒绝原因	<code>ShopProductMapper</code>
店铺	<code>myStore(user)</code> 、 <code>updateMyStore(request,user)</code> 、 <code>followStore(id,user)</code>	店铺归属、关注唯一性	<code>ShopStoreMapper</code>
订单评价	<code>createOrders(request,user)</code> 、 <code>orders(user)</code> 、 <code>reviewOrder(id,request,user)</code>	买家身份、商品存在、订单归属、不可重复评价	<code>ShopOrderMapper</code>
购物车	<code>list(token)</code> 、 <code>add(token,request)</code> 、 <code>updateQuantity</code> 、 <code>select</code> 、 <code>remove</code>	登录、商品存在、禁止自购、数量和购物车项归属	<code>CartMapper</code> 、 <code>ShopProductMapper</code>
收货地址	<code>list(token)</code> 、 <code>add</code> 、 <code>update</code> 、 <code>setDefault</code> 、 <code>remove</code>	登录、必填项、本人地址归属、默认地址唯一性	<code>AddressMapper</code>
话题帖子	<code>createTopic</code> 、 <code>createTopicPost</code> 、 <code>createTopicComment</code>	登录、内容非空、目标话题/帖子存在	<code>ShopTopicMapper</code>
社区动作	<code>toggleTopicPostLike</code> 、 <code>toggleTopicPostAction</code>	用户与动作组合唯一；再次操作执行取消	<code>ShopTopicMapper</code>

商品发布请求应包含名称、分类、描述、成色、故事、价格、最低价、场景、地区和图片。订单创建请求包含一个或多个订单项目；服务端不信任前端显示价格，而以商品数据重新确定买卖双方和交易金额。

4.7 聊天模块方法设计

所属类	方法	设计职责
ConversationService	createConversation(request,buyerId)	按买家、卖家、商品复用既有会话，否则创建新会话
ConversationService	validateAccess(user,conversation)	仅允许会话买家或卖家继续操作
ConversationService	updateStatus、markAllAsRead、deleteConversation	维护状态、已读和会话生命周期
ChatMessageService	createMessage(dto)	校验内容并写入消息记录
ChatMessageService	getMessages(covId)、countUnread(covId,userId)	获取历史消息和未读数
ChatMessageService	sendMessageAndBroadcast(dto)	保存成功后再广播，保证历史与实时消息一致
AiBargainService	getBargainSuggestion(covId,userId,role)	读取商品价格、底价和近期消息，形成议价建议
AiAssistantService	ask(question)	调用外部 AI；无密钥或调用失败时返回降级信息

5 关键模块动态设计

5.1 UC-01 注册并登录

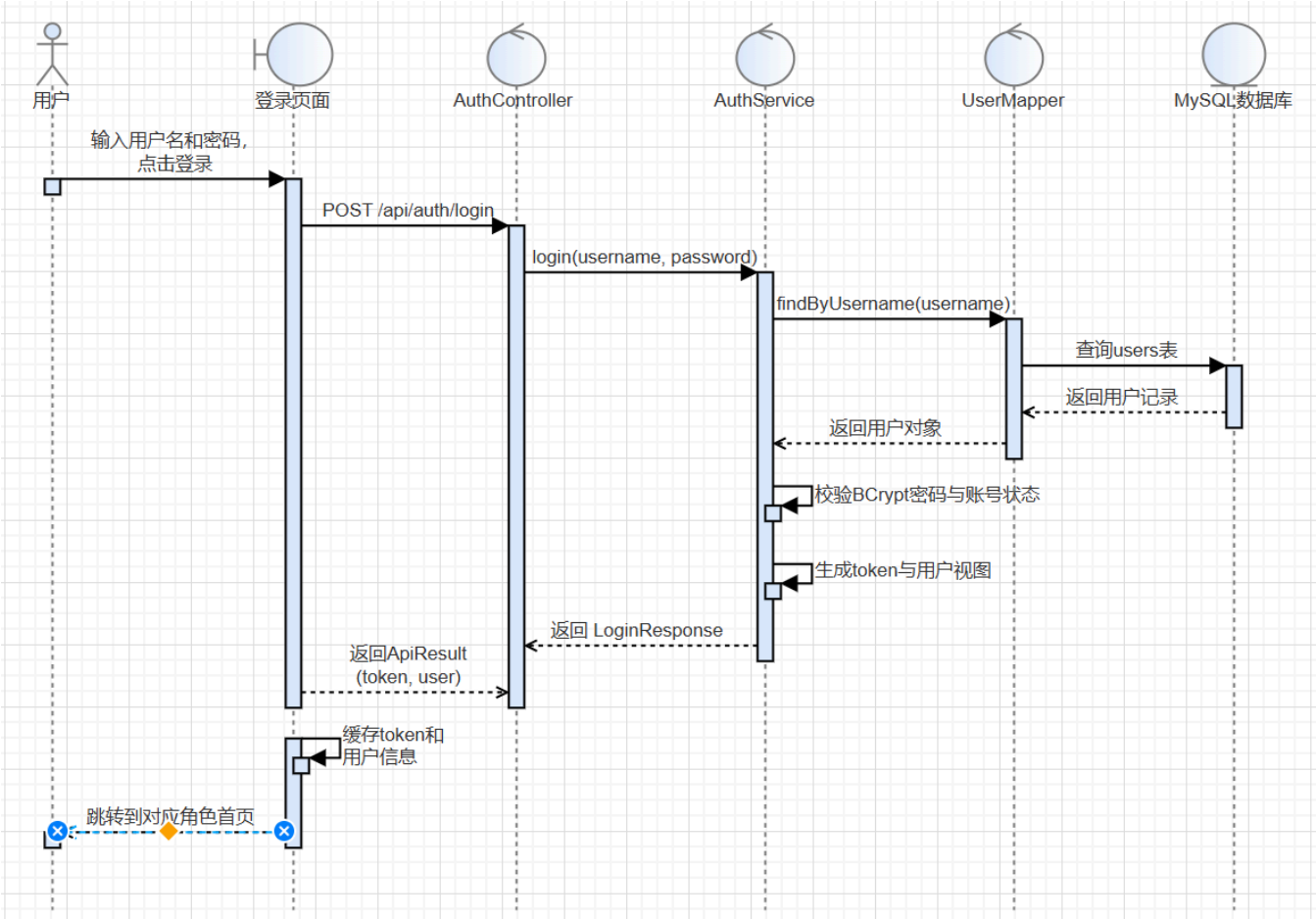


图 5-1 用户登录顺序图

用户提交账号密码后，登录页面调用 `POST /api/auth/login`；Controller调用 `AuthService.login`；`UserMapper` 查询用户；服务校验 `BCrypt` 密码与账号状态，生成随机 `Token` 和用户视图；前端缓存结果并按角色进入相应功能入口。用户不存在或密码错误返回 `401`，禁用账号返回 `403`。

5.2 UC-06 卖家发布商品与 UC-07 管理员审核商品

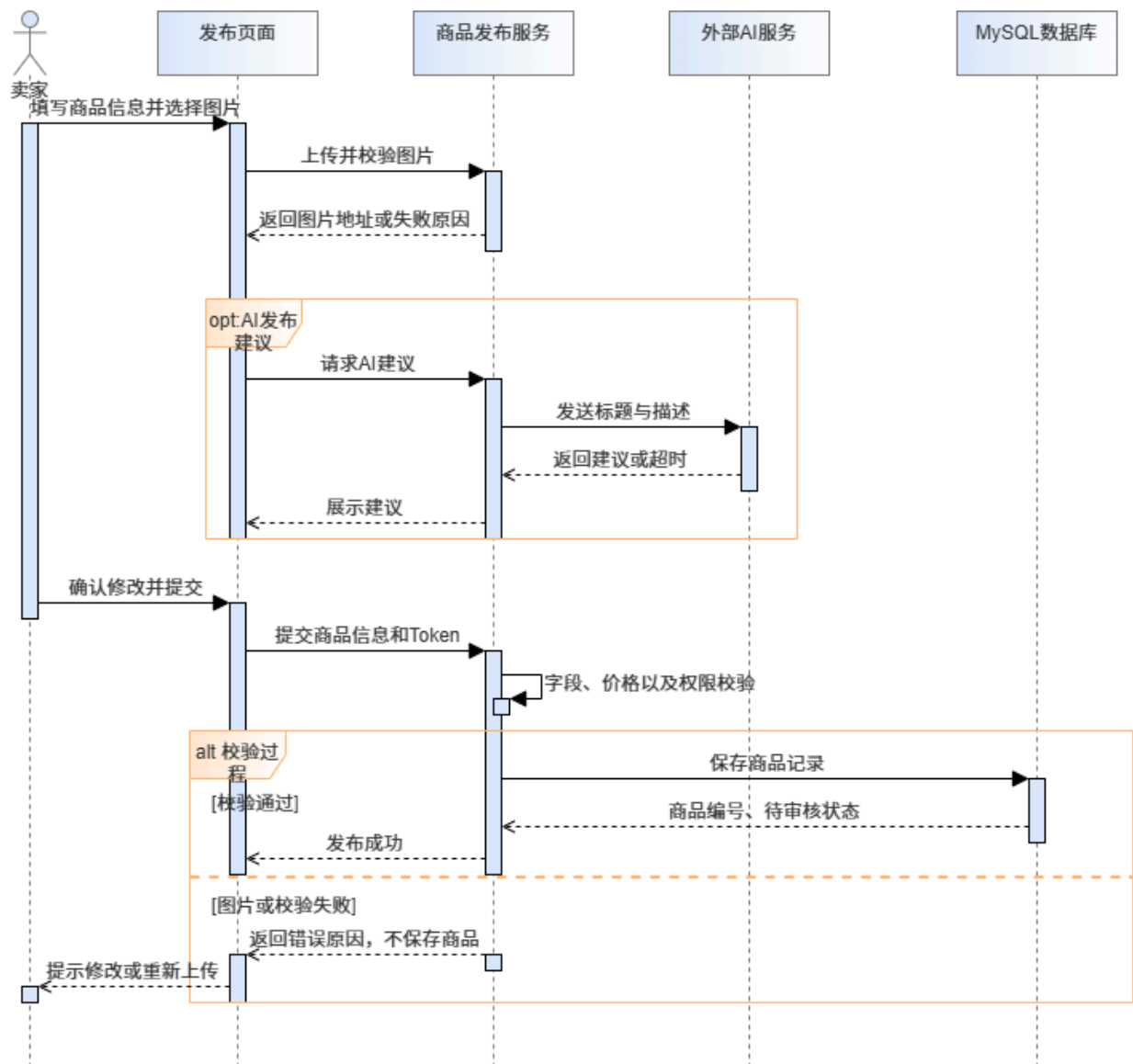


图 5-2 商品发布顺序图

顺序图描述卖家上传图片、可选请求 AI 建议、提交商品、完成字段/价格/权限校验并写入数据库的消息顺序。成功后返回商品编号和待审核状态；校验失败时不写入商品。

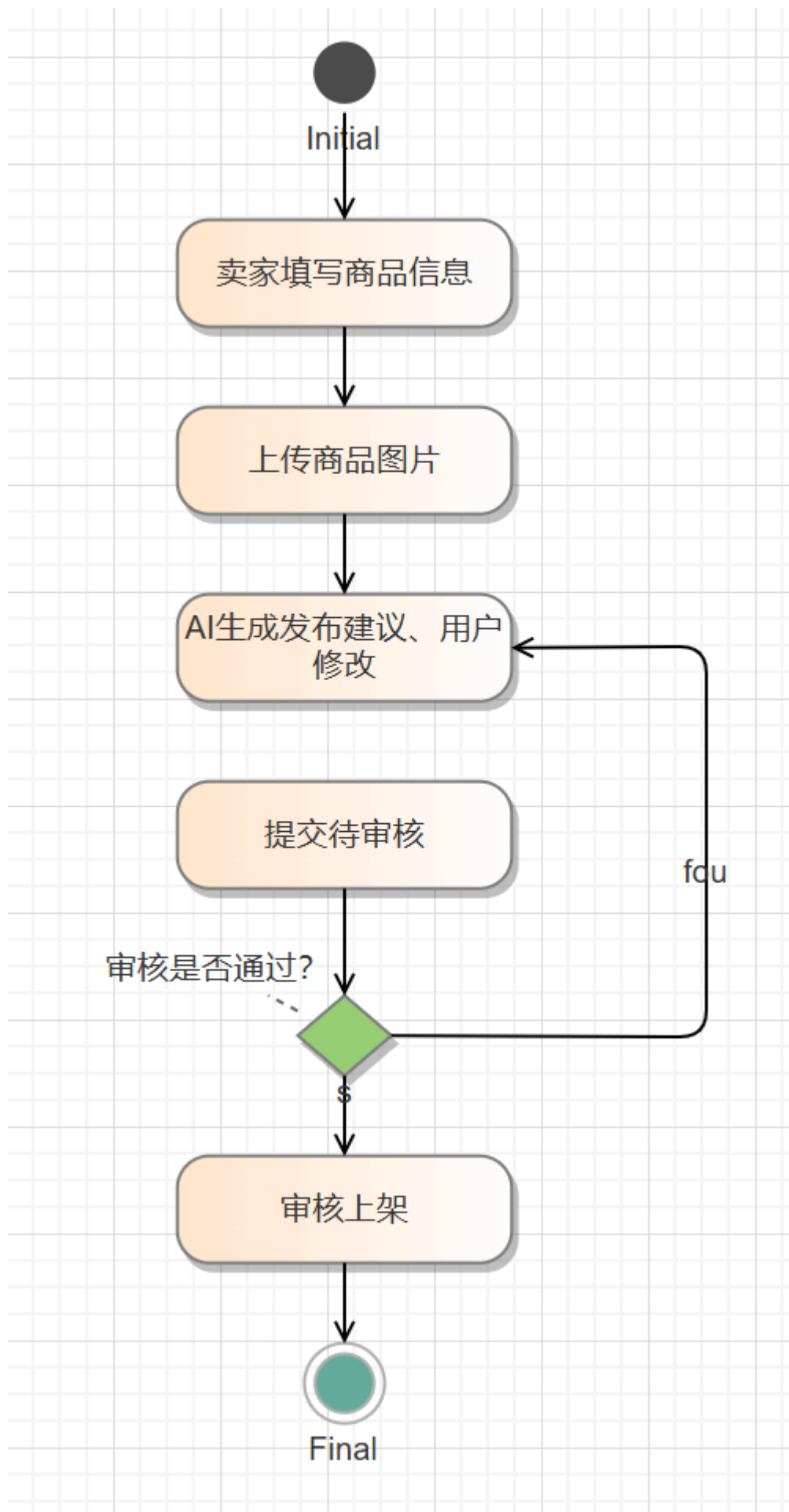


图 5-3 商品发布与审核活动图

卖家填写信息和上传图片，可使用 AI 建议并修改；提交后进入待审；审核通过后上架，审核不通过则返回修改后再次提交。活动图中的判断应在 CodeArts 中使用 Decision/Merge Node 表示。

5.3 UC-08 创建订单与 UC-09 查询评价

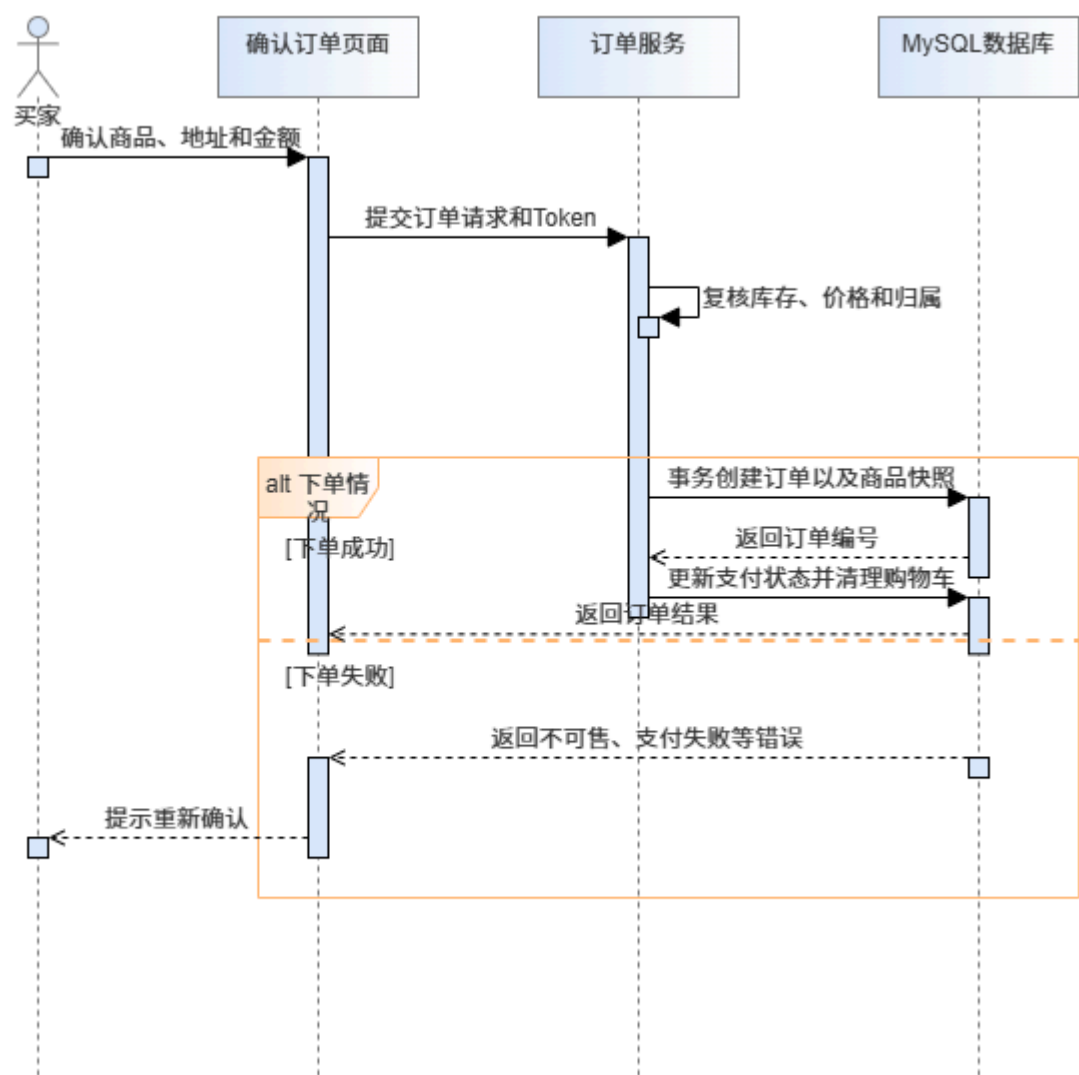


图 5-4 订单创建顺序图

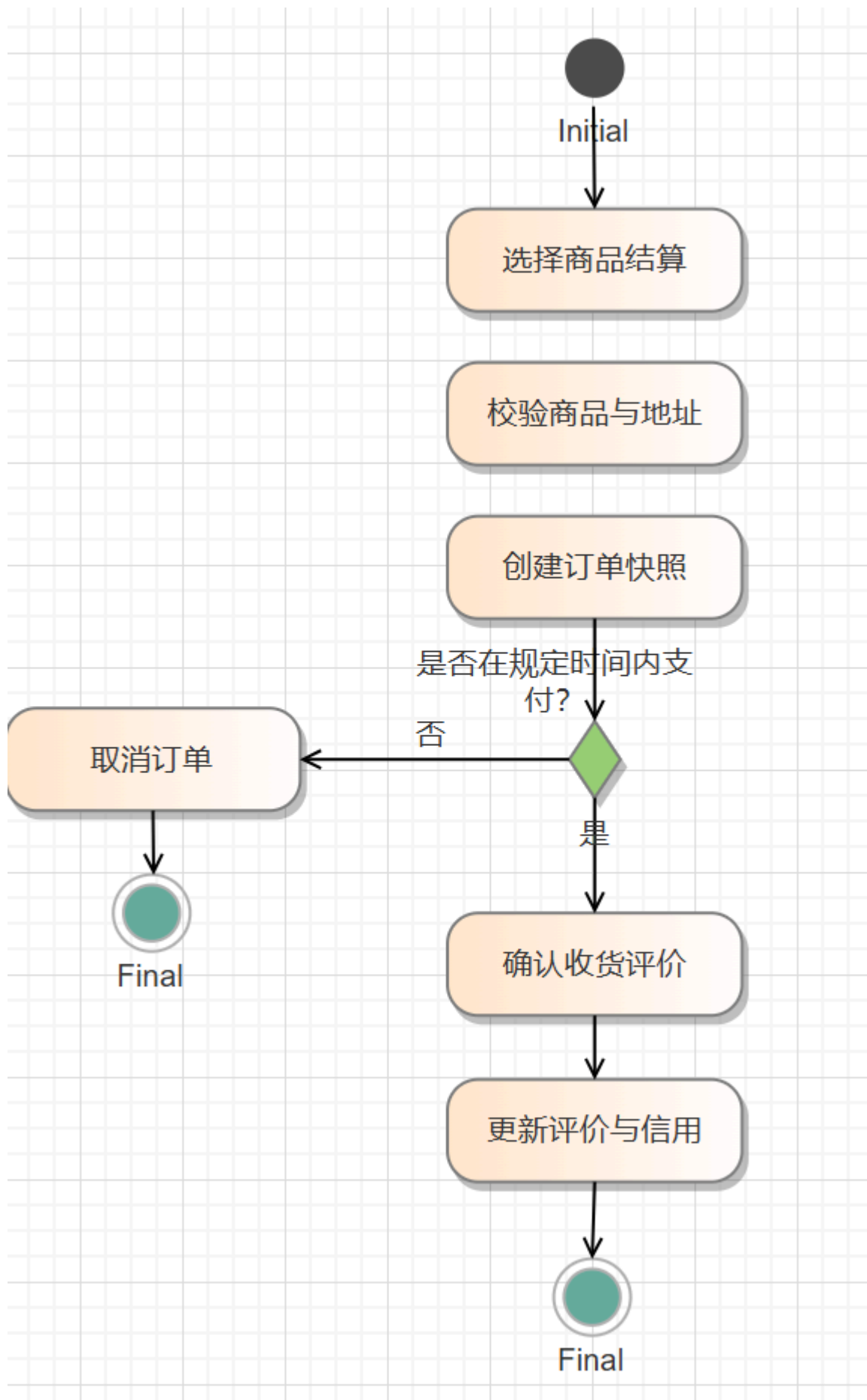


图 5-5 交易结算活动图

结算从选择商品开始，依次校验商品和地址、创建订单快照、处理支付判断、确认收货评价并更新信用。当前演示版未接入真实支付和物流，实际运行会跳过外部支付、发货与收货网关。

5.4 UC-11 建立会话、收发消息并使用 AI 议价

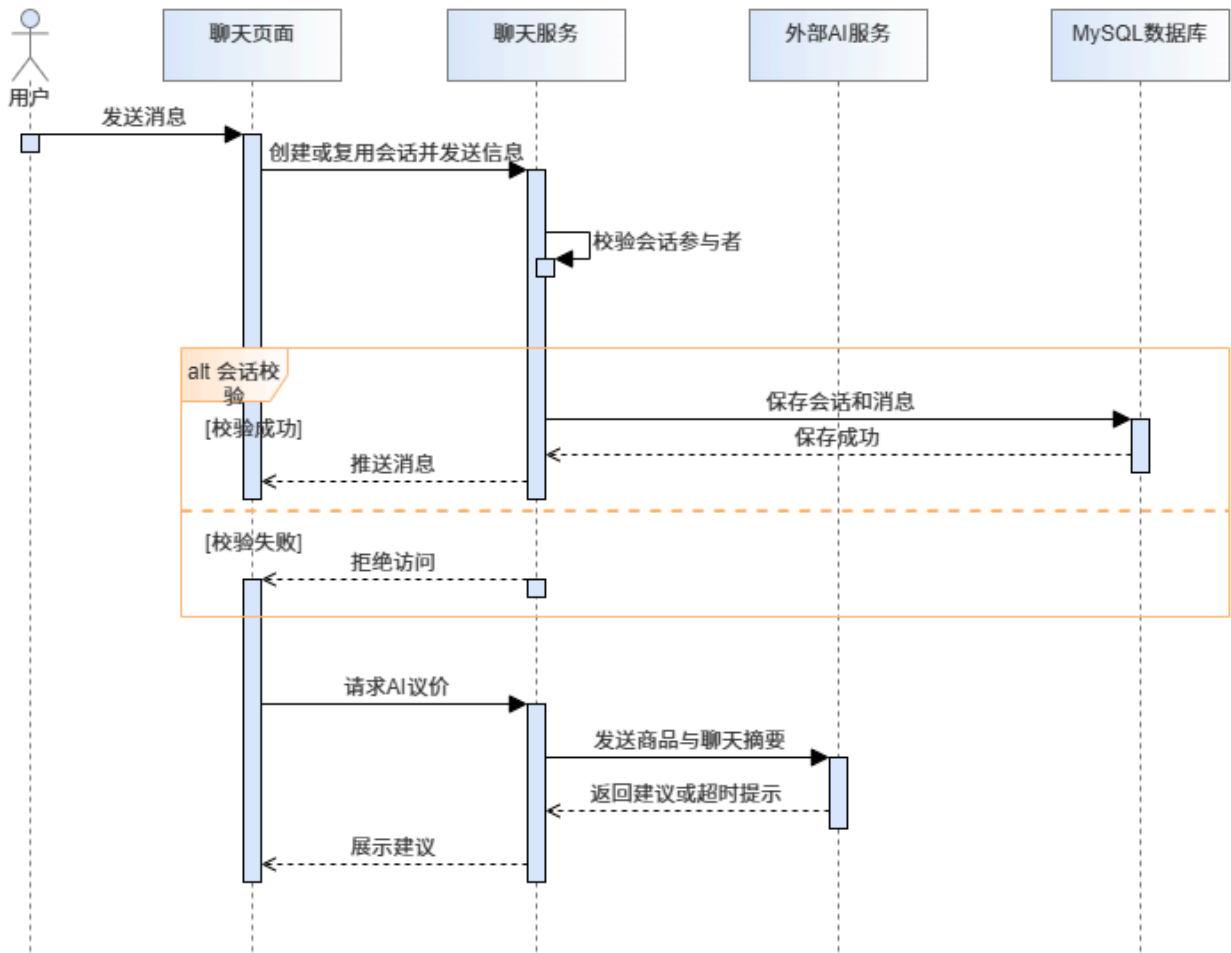


图 5-6 聊天与 AI 议价顺序图

聊天页面请求创建或复用会话，服务校验当前用户是否为参与者，保存会话和消息后通过 WebSocket 推送。请求 AI 议价时，服务将商品与聊天摘要发送给外部 AI，返回建议或降级提示。

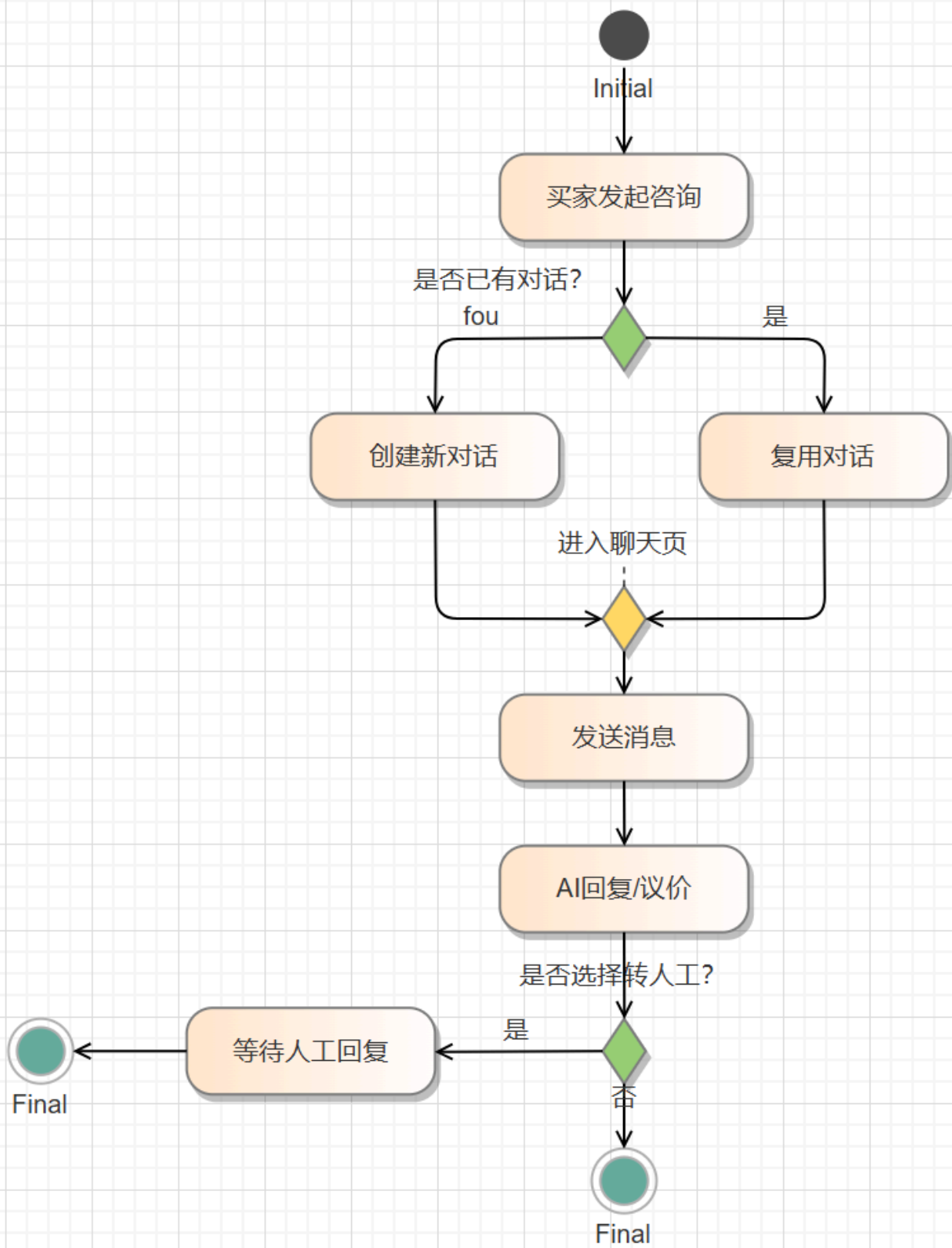


图 5-7 聊天议价活动图

系统先判断是否存在相同买家、卖家和商品组合的会话，再创建或复用会话。消息发送后可获得 AI 回复；选择转人工时，会话状态切换为 `active` 并等待卖家回复。

5.5 UC-06、UC-07 商品状态设计

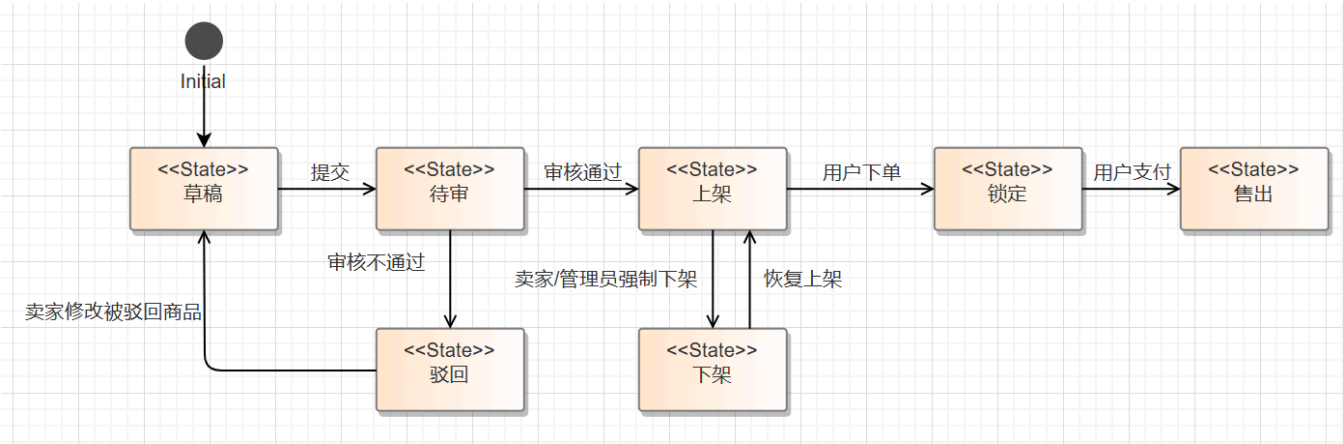


图 5-8 商品状态图

UML 状态	当前代码状态	说明
草稿	前端未提交数据	当前数据库不保存草稿
待审	pending	等待管理员审核
上架	approved	进入公开列表
驳回	rejected	保存拒绝原因
下架	offline	设计保留
锁定、售出	尚未完整实现	需与订单状态联动

5.6 UC-08、UC-09 订单目标状态设计

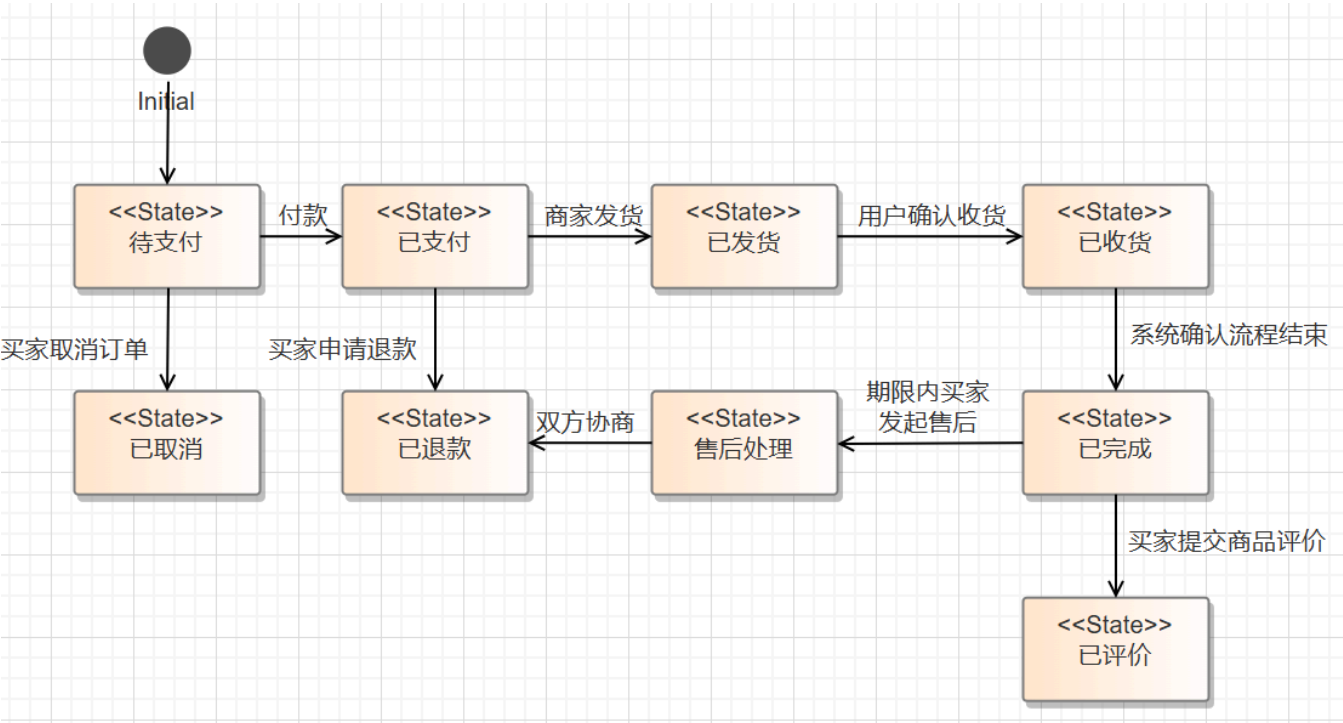


图 5-9 订单状态图

完整订单状态机包括待支付、已支付、已发货、已收货、已完成、已评价、已取消、售后处理和已退款，属于明确标注的目标模型。当前 `shopService.createOrders` 在插入时直接把状态写为 `completed`；取消接口只能从当前状态改为 `cancelled`，评价通过独立记录表示。

5.7 UC-14 用户账号状态设计

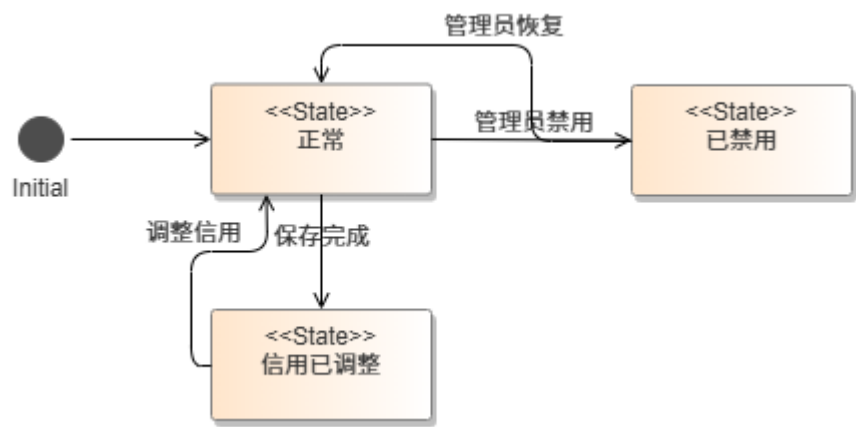


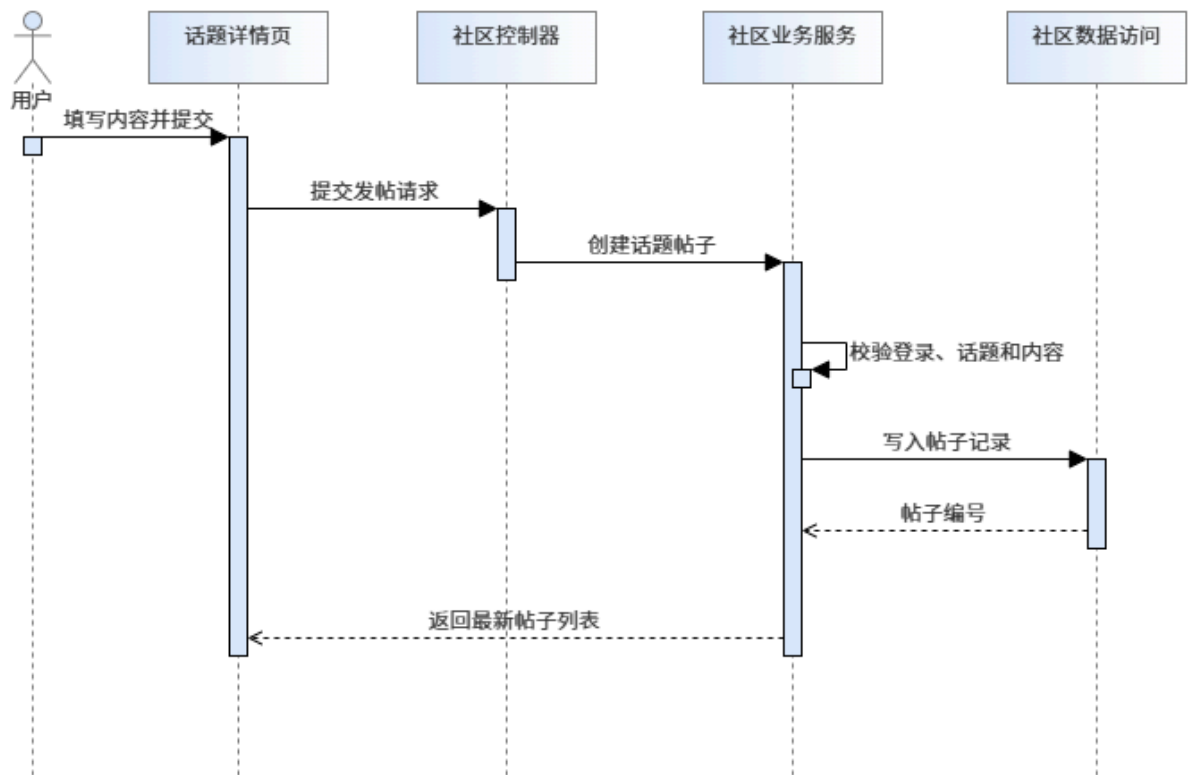
图 5-10 用户账号治理状态机图

用户账号在治理模块中经历正常、禁用、恢复等状态，信用分可随评价或管理员操作调整。禁用状态的账号在下次认证请求时被拒绝。

5.8 UC-10 创建话题并完成社区互动

社区发帖不是单纯的数据插入。服务端依次完成身份识别、话题存在性检查、内容检查、可选商品/店铺关联检查、帖子写入和最新列表重载。

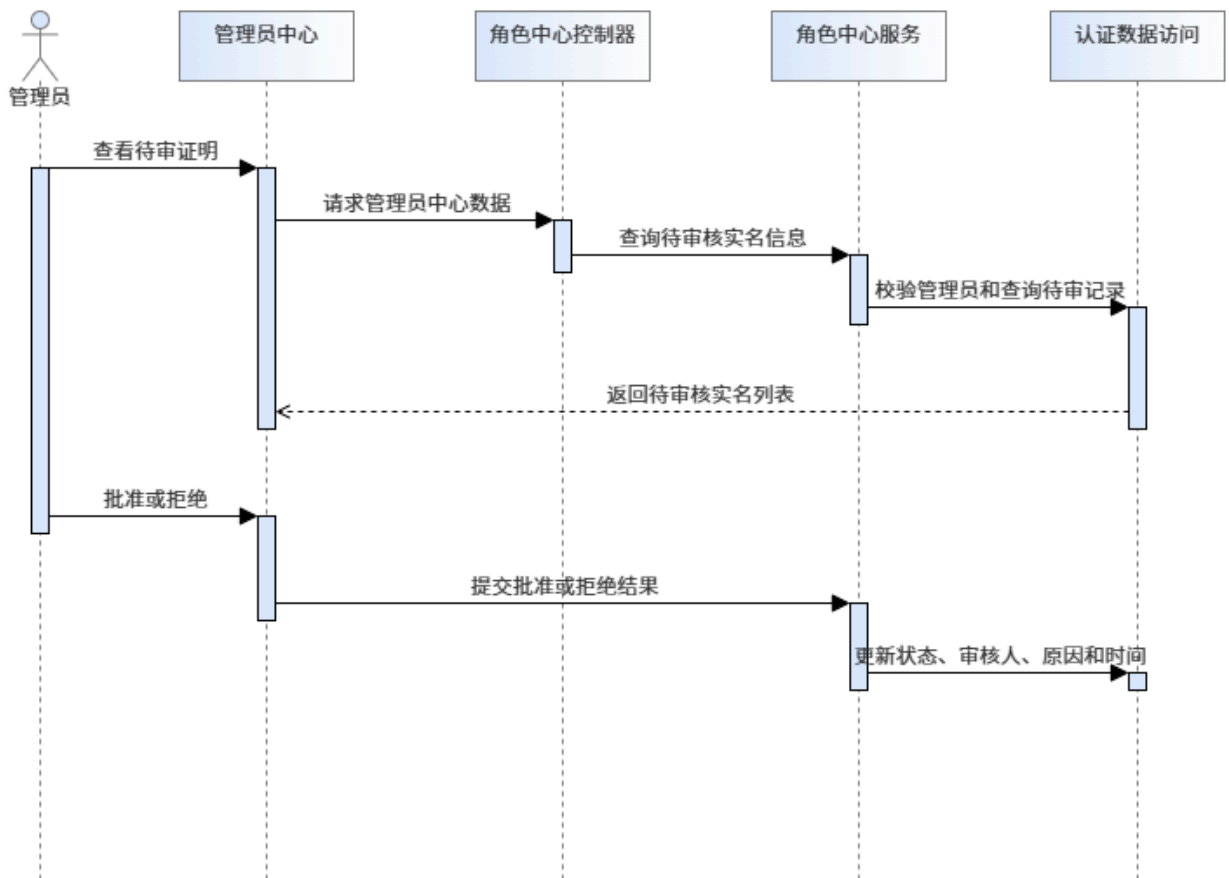
步骤	调用	正常结果	异常分支
1	页面POST <code>/api/topics/{id}/posts`</code>	Controller 获得话题编号和请求体	未登录返回 401
2	<code>ShopService.createTopicPost</code>	解析当前用户和目标话题	话题不存在返回 404
3	校验 <code>content</code> 、 <code>productId</code> 、 <code>storeId</code>	形成帖子记录	内容为空或关联目标无效返回 400
4	<code>ShopTopicMapper</code> 写入 <code>topic_post</code>	返回帖子编号	数据库写入失败回滚本次请求
5	重新查询话题帖子	返回 <code>List<TopicPostView></code>	查询失败由统一异常处理返回 500



5.9 UC-12 提交实名认证并由管理员审核

实名认证申请只保存业务需要的脱敏信息。管理员中心查询待审记录后，可执行批准或拒绝；审核动作必须记录审核人和审核时间，拒绝时还要记录原因。

操作	Service 方法	数据变化	权限与异常
提交 实名	<code>submitRealName / submitSellerRealName</code>	新增 <code>pending</code> 记录	登录用户；重复待审申请应被拒绝
取消 申请	<code>cancelRealName / cancelSellerRealName</code>	取消当前待审申请	只能操作本人申请
批准	<code>approveRealName(token, id)</code>	<code>status=approved</code> ，记录审核人和时间	仅管理员；记录不存在返回 404
拒绝	<code>rejectRealName(token, id, request)</code>	<code>status=rejected</code> ，保存原因、审核人和时间	仅管理员；拒绝原因不能为空



5.10 现有对象级动态模型覆盖

用例	当前可引用的详细设计图	判定
UC-01 注册并登录	图 5-1 登录顺序图	已覆盖
UC-02 浏览、搜索并查看商品	无独立对象级动态图	未覆盖
UC-03 收藏商品、关注店铺并查看	无独立对象级动态图	未覆盖
UC-04 维护个人资料与头像	无独立对象级动态图	未覆盖
UC-05 卖家查看并维护店铺	无独立对象级动态图	未覆盖
UC-06 卖家发布并维护商品	图 5-2、图 5-3	已覆盖
UC-07 管理员审核商品	图 5-3、图 5-8	部分覆盖，审核对象调用未单独展开
UC-08 维护购物车和地址并创建订单	图 5-4、图 5-5	已有目标模型；当前实现差距已标注
UC-09 查询订单并提交评价	图 5-5、图 5-9	部分覆盖，评价对象调用未单独展开
UC-10 创建话题并完成社区互动	社区发帖顺序图	已覆盖主要发帖流程

用例	当前可引用的详细设计图	判定
UC-11 建立会话、收发消息并使用 AI 议价	图 5-6、图 5-7	已覆盖
UC-12 提交实名认证并由管理员审核	实名认证审核顺序图	已覆盖
UC-13 使用 AI 购物助手	概要设计图 4-8	只有组件级图，对象级未覆盖
UC-14 管理员维护用户状态与信用	图 5-10 用户账号治理状态机图	覆盖状态变化，具体对象调用未展开

6 数据设计

6.1 用户与店铺数据

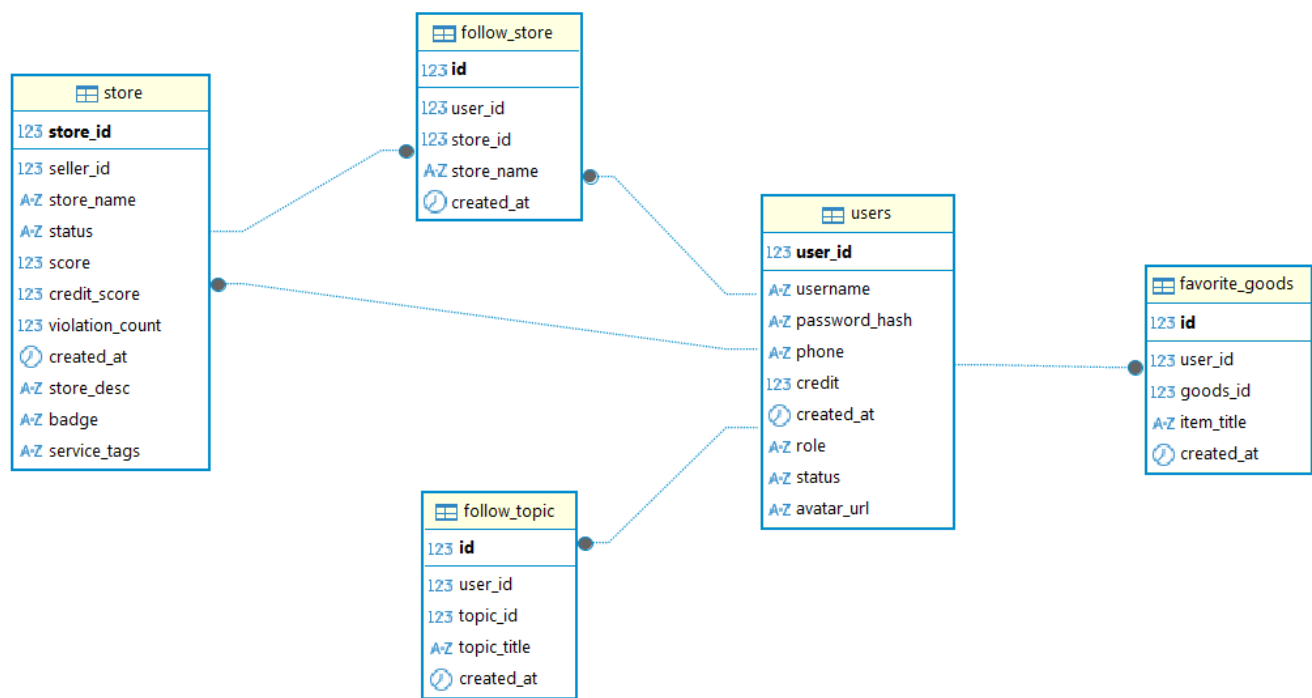


图 6-1 用户与店铺关系图

users 保存账号、密码散列、手机号、信用、角色、状态和头像；**store** 通过 **seller_id** 关联卖家；关注和收藏表记录用户与目标对象的多对多关系。

6.2 商品、订单与评价数据

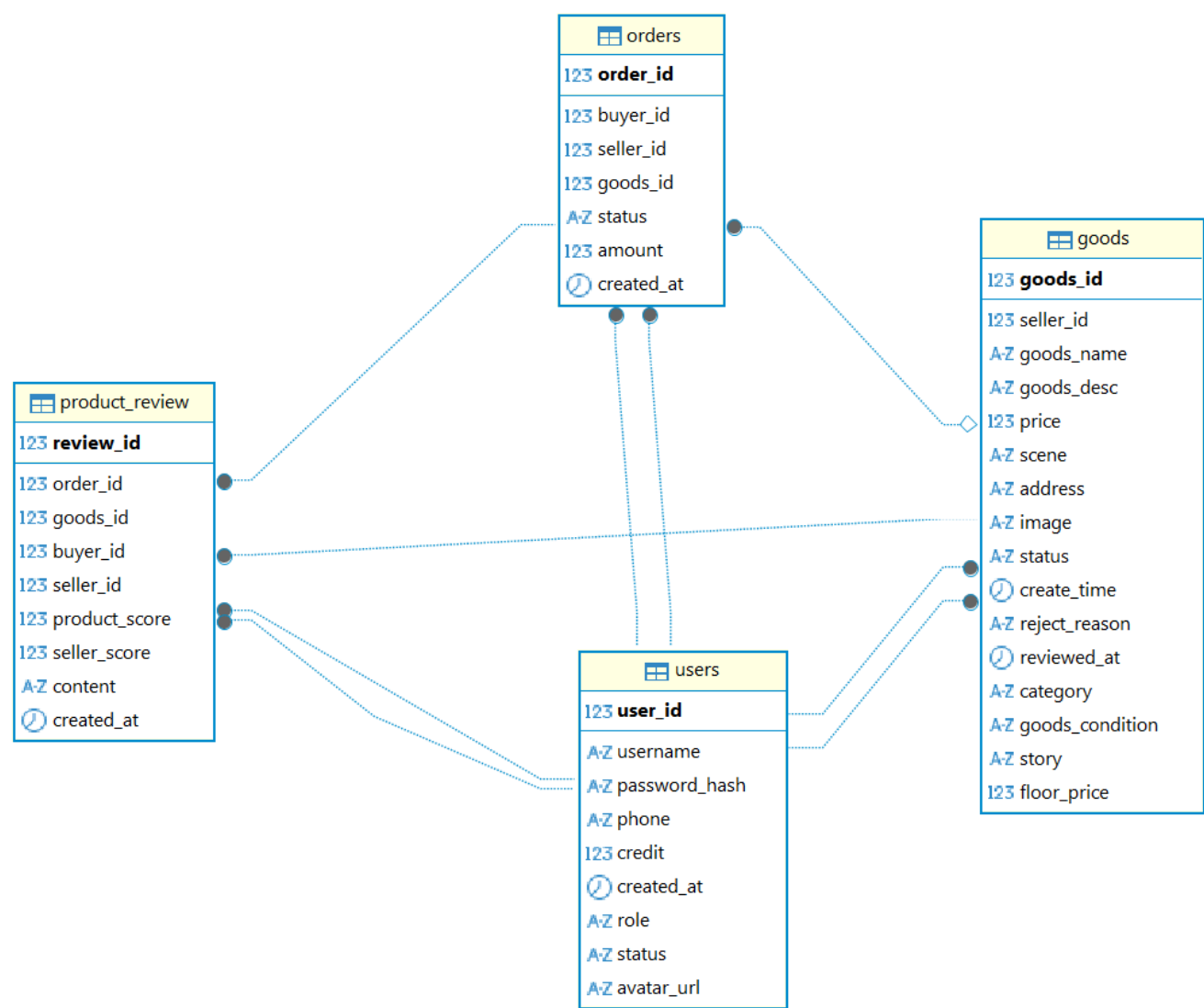


图 6-2 交易核心 ER 图

`goods` 通过 `seller_id` 关联卖家；`orders` 保存买家、卖家和商品编号；`product_review` 保存订单、商品、买卖双方、双评分和文字评价。`product_review.order_id` 唯一，价格和金额使用 `DECIMAL(10,2)`。

6.3 社区与聊天数据

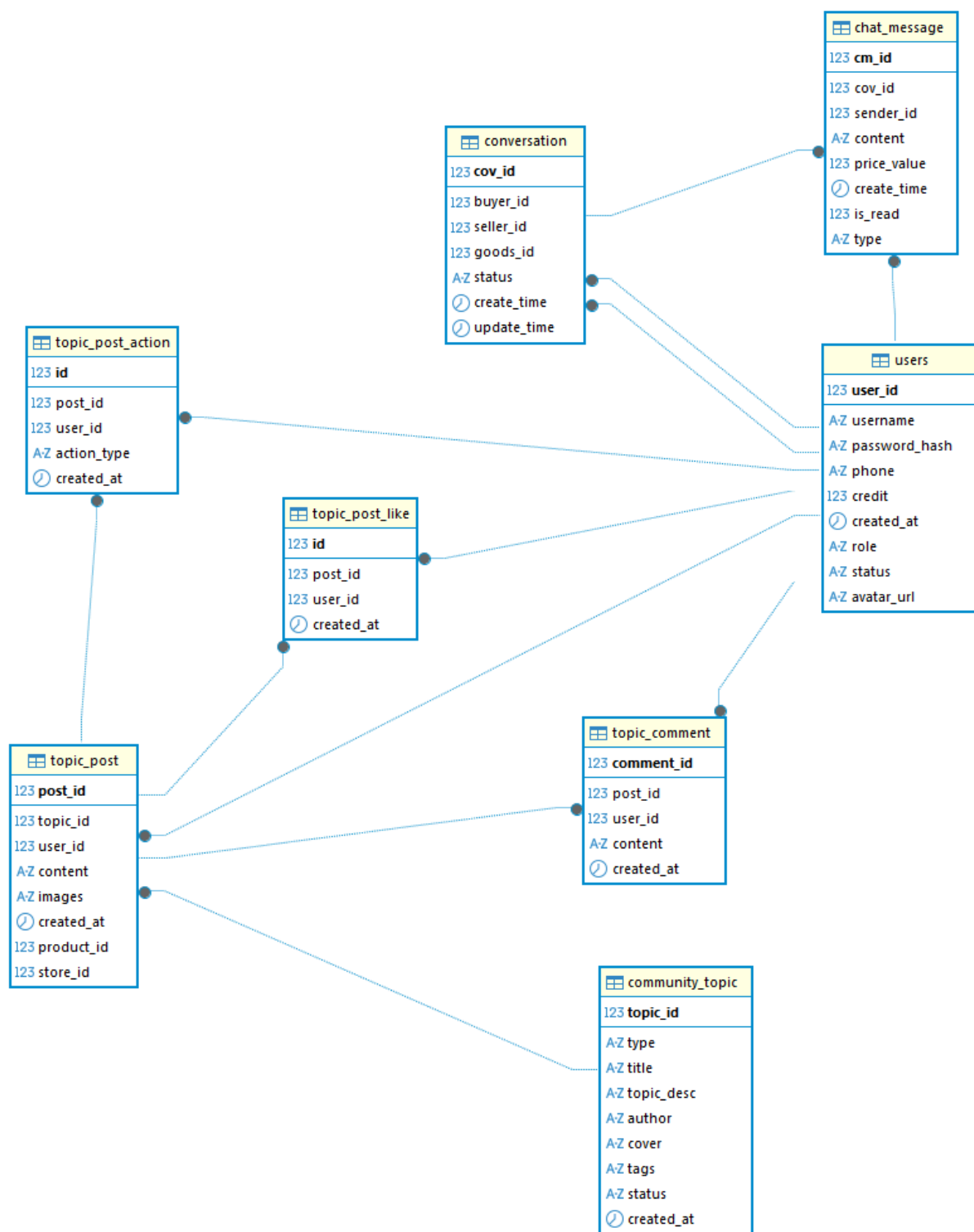


图 6-3 社区与聊天关系图

社区以 `community_topic` 为主题，帖子属于话题并由用户发布，评论属于帖子；点赞与其他动作使用独立关系表。聊天由 `conversation` 和 `chat_message` 组成，每条消息记录会话、发送者、内容、报价、类型、时间和已读标记。

6.4 数据约束与索引

表/字段	约束或索引	目的
product_review(order_id)	UNIQUE	防止重复评价
follow_store(user_id, store_id)	UNIQUE	防止重复关注店铺
follow_topic(user_id, topic_id)	UNIQUE	防止重复关注话题
topic_post_like(post_id, user_id)	UNIQUE	防止重复点赞
topic_post_action(post_id, user_id, action_type)	UNIQUE	区分并约束互动
chat_message(cov_id, create_time)	INDEX	按会话时间查询消息
conversation(update_time)	INDEX	按最近活跃时间排序

数据库统一使用 utf8mb4。当前脚本未普遍建立外键，关联完整性主要由 Service 校验。

6.5 核心表字段设计

6.5.1 users 用户表

字段	类型	约束/默认值	说明
user_id	INT	PK, AUTO_INCREMENT	用户编号
username	VARCHAR(50)	NOT NULL	登录名，业务层保证唯一
password_hash	VARCHAR(255)	NOT NULL	BCrypt 密码散列
phone	VARCHAR(20)	NULL	手机号，对外展示时脱敏
credit	INT	DEFAULT 100	用户信用分
role	VARCHAR(20)	DEFAULT buyer	buyer、seller、admin
status	VARCHAR(20)	DEFAULT normal	normal 或 disabled
avatar_url	VARCHAR(500)	NULL	/files/ 或 HTTP(S) 地址
created_at	TIMESTAMP	CURRENT_TIMESTAMP	创建时间

6.5.2 goods 商品表

字段	类型	约束/默认值	说明
goods_id	INT	PK, AUTO_INCREMENT	商品编号
seller_id	INT	NOT NULL	发布卖家编号
goods_name	VARCHAR(255)	NOT NULL	商品名称

字段	类型	约束/默认值	说明
category	VARCHAR(100)	NULL	商品分类
goods_desc	TEXT	NULL	商品描述
goods_condition	VARCHAR(100)	NULL	成色
story	TEXT	NULL	商品故事
price	DECIMAL(10,2)	DEFAULT 0	售价
floor_price	DECIMAL(10,2)	NULL	AI 议价底价，不直接对买家展示
scene	VARCHAR(20)	DEFAULT used	新品/二手场景
address	VARCHAR(255)	NULL	交易地区
image	TEXT	NULL	图片地址或地址集合
status	VARCHAR(20)	DEFAULT approved	审核与上架状态
reject_reason	VARCHAR(255)	NULL	审核拒绝原因
reviewed_at	DATETIME	NULL	审核时间
create_time	DATETIME	CURRENT_TIMESTAMP	发布时间

6.5.3 orders 与 product_review

表	关键字段	约束	设计说明
orders	order_id、buyer_id、seller_id、goods_id、status、amount、created_at	order_id 主键	保存交易主体、商品、状态和金额
product_review	review_id、order_id、goods_id、buyer_id、seller_id、product_score、seller_score、content	order_id 唯一	一个订单最多产生一条评价

评分在服务端校验合法区间，评价提交前校验订单归属和状态。评分统计由查询计算，避免客户端直接修改店铺评分。

6.5.4 conversation 与 chat_message

表	字段	类型/约束	说明
conversation	cov_id	INT PK	会话编号
conversation	buyer_id、seller_id、goods_id	INT NOT NULL	会话三方业务键
conversation	status	VARCHAR(20)	pending、ai、active

表	字段	类型/约束	说明
conversation	create_time、update_time	DATETIME	创建及最近活跃时间
chat_message	cm_id、cov_id、sender_id	INT	消息、会话与发送者编号
chat_message	content	TEXT NOT NULL	消息正文
chat_message	price_value	DECIMAL(10,2)	可选报价
chat_message	is_read	TINYINT(1) DEFAULT 0	已读标记
chat_message	type	VARCHAR(30)	普通、AI、报价或系统消息类型

6.5.5 社区与互动表

表	主键	关键业务字段	说明
community_topic	topic_id	type、title、topic_desc、cover、tags、status	话题主数据
topic_post	post_id	topic_id、user_id、content、images、product_id、store_id	帖子可关联商品或店铺
topic_comment	comment_id	post_id、user_id、content	帖子评论
topic_post_like	id	post_id、user_id	点赞关系，组合唯一
topic_post_action	id	post_id、user_id、action_type	收藏、分享等动作
follow_topic	id	user_id、topic_id、topic_title	话题关注

6.5.6 cart_item 与 user_address

表	关键字段	约束与用途
cart_item	cart_id、user_id、goods_id、quantity、selected、created_at	user_id + goods_id 唯一；所有修改操作同时按 cart_id 与 user_id 校验归属
user_address	address_id、user_id、receiver、phone、province、city、district、detail、is_default	地址按用户查询；首个地址自动默认，切换默认地址时先清除该用户原默认项

6.6 数据一致性与事务边界

- 1. 注册时“检查用户名 - 写入用户”应位于同一业务边界，并由数据库唯一索引作为最终防线。

- 2. 发布商品时图片上传与商品写入是两个步骤；商品写入失败不删除已上传文件，需由定期清理任务处理孤立文件。
- 3. 创建订单时必须重新读取商品并确定卖家和金额，不能直接信任前端快照。
- 4. 评价写入与信用更新应放在同一事务；当前若仅完成评价写入，应避免返回“信用已更新”的错误承诺。
- 5. 发送消息采用“先持久化、后推送”，推送失败时历史记录仍可通过 HTTP 重新读取。
- 6. 审核操作需同时更新状态、原因和审核时间；任何一步失败时保持原状态。

7 接口与公共机制设计

7.1 接口规范

- REST 基础路径为 `/api`，JSON 使用 UTF-8。
- 图片上传使用 `multipart/form-data`。
- 受保护接口使用 `Authorization: Bearer <token>`。
- 成功响应为 `{"code":0,"message":"ok","data":...}`。
- 业务失败同时使用适当 HTTP 状态码和 `ApiResult` 错误码。

7.2 主要 REST 接口

模块	方法与路径	作用
认证	<code>POST /api/auth/register</code> 、 <code>POST /api/auth/login</code> 、 <code>GET /api/auth/me</code>	注册、登录、当前用户
角色中心	<code>GET /api/center/buyer seller admin</code>	不同角色中心数据
商品	<code>GET/POST /api/products</code> 、 <code>GET /api/products/{id}</code>	查询、发布和查看商品
店铺	<code>GET /api/stores</code> 、 <code>GET/PUT /api/stores/mine</code>	店铺查询和维护
订单	<code>GET/POST /api/orders</code> 、 <code>POST /api/orders/{id}/review</code>	订单与评价
审核	<code>GET /api/admin/audit</code> 、 <code>POST /api/admin/audit/{id}</code>	商品审核
社区	<code>/api/topics</code> 、 <code>/api/topics/{id}/posts</code> 、 <code>/api/topic-posts/{id}/...</code>	话题和互动
聊天	<code>/api/chat/conversations/**</code>	会话、消息、转人工和议价
AI	<code>/api/ai/assistant</code> 、 <code>/api/ai/assist</code> 、 <code>/api/ai/publish-suggestion</code>	AI 辅助
上传	<code>POST /api/upload/image</code>	返回 <code>/files/...</code>

7.3 WebSocket 设计

WebSocket 端点为 `/ws`，使用 SockJS 和 STOMP。客户端在 CONNECT 阶段携带 Token，服务端设置用户 Principal；应用消息发送到 `/app`，服务端通过 `/queue` 或 `/topic` 推送。消息先写入 `chat_message`，再向目标用户推送；已读回执更新数据库后发送通知。

7.4 认证、异常与上传

`AuthService` 使用 BCrypt 校验密码，并在内存中维护随机 Token。`GlobalExceptionHandler` 统一处理参数校验、请求体错误、缺参、业务状态异常和未知异常。上传目录通过 `/files/**` 映射为静态资源，上传接口检查空文件、类型、扩展名和大小。

HTTP 状态	含义	前端处理
400	参数或请求格式错误	显示字段问题
401	未登录或 Token 失效	清理会话并登录
403	无权限	提示拒绝访问
404	资源不存在	显示空状态
409	唯一性冲突	提示重复操作
500	服务异常	显示通用失败并记录日志

7.5 关键接口输入输出设计

7.5.1 登录接口

项目	内容
方法与路径	<code>POST /api/auth/login</code>
请求体	<code>username: String</code> ， <code>password: String</code>
成功数据	<code>token: String</code> ， <code>user: AuthUserView</code>
业务规则	用户存在、密码匹配、状态不是 <code>disabled</code>
失败	400 请求无效；401 账号或密码错误；403 账号禁用

7.5.2 发布商品接口

项目	内容
方法与路径	<code>POST /api/products</code>
认证	Bearer Token，且当前用户为卖家
请求体	名称、分类、描述、成色、故事、价格、最低价、场景、地区、图片
成功数据	<code>ProductView</code> ，包含商品编号和审核状态

项目	内容
业务规则	必填字段非空；价格合法；最低价不高于售价；新商品写入 <code>pending</code>
失败	400 字段错误；401 未登录；403 非卖家；500 保存失败

7.5.3 创建订单接口

项目	内容
方法与路径	<code>POST /api/orders</code>
认证	Bearer Token，当前用户作为买家
请求体	<code>CreateOrderRequest</code> ，包含订单项目集合
成功数据	<code>List<OrderView></code>
业务规则	商品存在且可交易；不能购买自己的商品；金额由服务端商品记录确定
失败	400 项目为空或商品无效；401 未登录；409 状态冲突

7.5.4 创建会话接口

项目	内容
方法与路径	<code>POST /api/chat/conversations</code>
请求体	<code>sellerId: Integer</code> 、 <code>goodsId: Integer</code> 、可选 <code>status</code>
成功数据	<code>Conversation</code>
业务规则	当前用户作为买家；卖家和商品必须存在；优先复用相同三方组合的会话
失败	401 未登录；404 商品或卖家不存在；403 身份不合法

7.5.5 商品审核接口

项目	内容
方法与路径	<code>POST /api/admin/audit/{id}</code>
认证	管理员 Token
请求体	审核动作及可选拒绝原因
成功数据	<code>AuditResult</code>
业务规则	通过写 <code>approved</code> ；拒绝写 <code>rejected</code> 和原因；记录审核时间
失败	400 动作或原因无效；403 非管理员；404 商品不存在

7.6 统一响应与错误传播

`ApiResponse<T>` 只包含 `code`、`message` 和 `data`。业务成功使用 `code=0`；错误响应的 HTTP 状态与 `code` 保持一致。Controller 不捕获并吞掉业务异常，`ResponseStatusException` 由 `GlobalExceptionHandler` 转换为统一失败结果。前端 `request.js` 仅把 2xx 视为成功，并将状态码与响应体一并交给页面错误分支。

参数验证分为三层：DTO 注解验证结构与长度，Service 验证业务规则，数据库唯一约束保证并发条件下的最终一致性。错误信息不得包含密码散列、数据库密码、AI 密钥、完整身份证号或内部 SQL。

8 部署、测试与实现边界

8.1 部署架构

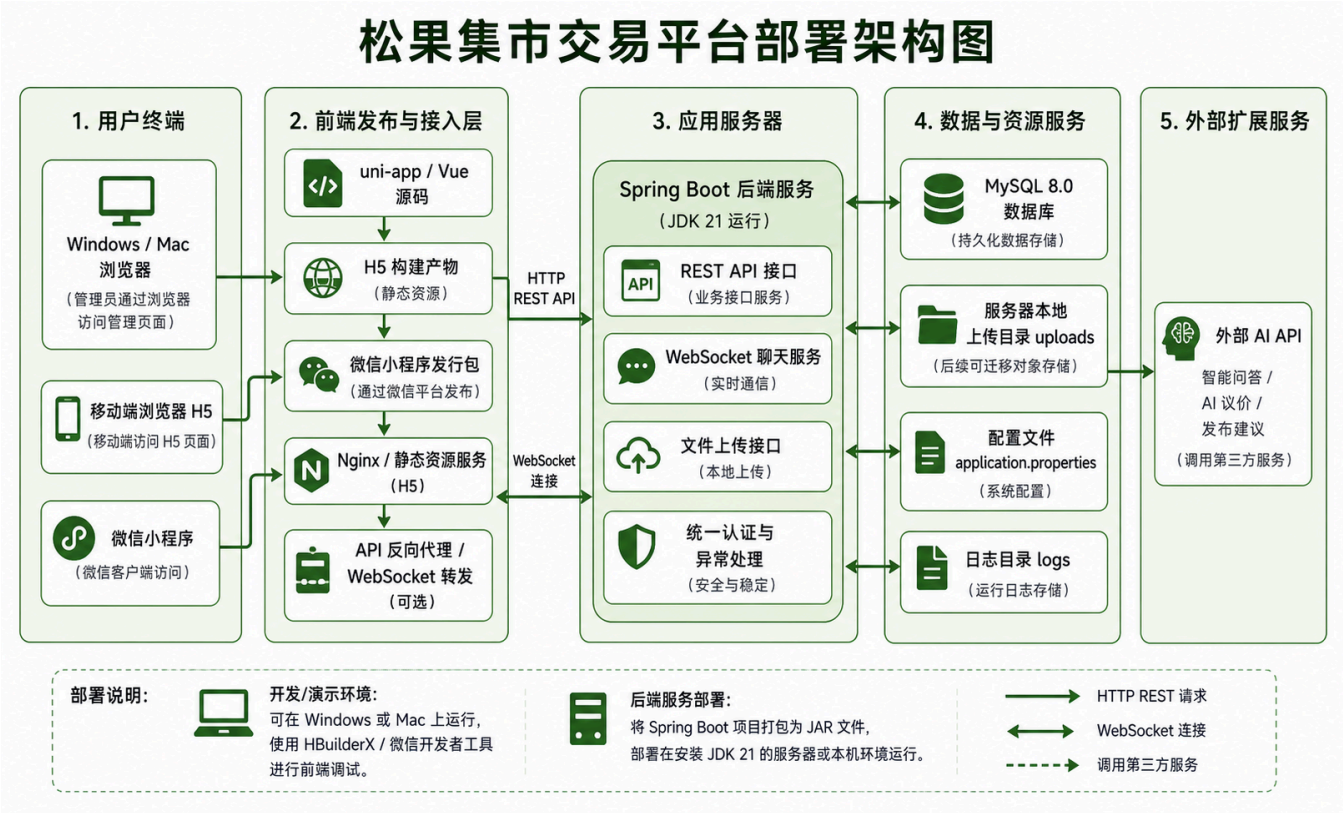


图 8-1 松果集市部署架构图

部署架构图描述用户终端、前端接入、Spring Boot 服务、MySQL/上传资源和外部 AI API 的关系。单机一键部署采用 Docker Compose：Nginx 提供 H5 并转发 `/api`、`/files` 和 `/ws`；后端连接 MySQL；上传和数据库目录由数据卷持久化。集群部署使用 `k8s/` 中的 Namespace、ConfigMap、MySQL、backend、frontend 和 Kustomize 清单，并由 `scripts/deploy-k8s.*` 执行。

8.2 容器运行设计

容器	职责	对宿主机端口
frontend	Nginx 静态资源和反向代理	暴露 HTTP_PORT
backend	Spring Boot REST、WebSocket 和文件服务	不直接暴露
mysql	MySQL 8.4	不直接暴露

MySQL 健康后启动 backend，backend 健康后 frontend 对外服务。`deploy.env` 保存本机端口、数据库密码、公开来源和可选 AI 参数，不提交版本库。

8.3 测试重点

- 1. 四类参与者与接口权限一致。
- 2. 登录顺序图的消息能通过接口与日志验证。
- 3. 商品发布后先待审，通过后才进入公开列表。
- 4. 非参与者无法访问会话。
- 5. 订单只能由所属买家评价，重复评价被阻止。
- 6. AI 不可用时普通商品、订单和聊天功能仍正常。
- 7. 一键部署后三个容器正常，图片经 `/files` 显示。
- 8. Kubernetes 清单可由 Kustomize 渲染，部署脚本能够等待工作负载和健康检查完成。

8.4 实现约束与版本边界

本版本采用以下明确约束，测试和答辩均以此为准：

- 1. 运行组件固定为 Nginx、Spring Boot 和 MySQL；可使用 Docker Compose 或 Kubernetes 承载，总体图中的 Redis 是扩展接口，不属于本版本运行依赖。
- 2. 商品持久化状态使用 `pending`、`approved`、`rejected` 和 `offline`；状态图中的草稿、锁定、售出是完整领域模型，用于后续订单联动设计。
- 3. 本版本订单用于演示创建与评价闭环，不接入第三方支付、物流、退款和售后网关。
- 4.
- 5. 登录 Token 由单实例后端内存维护，服务重启后用户重新登录。
- 6. AI 是可选增强服务，未配置密钥或调用失败时使用降级提示，核心交易不依赖 AI 成功。

8.5 部署配置项设计

配置类别	配置项	使用位置	约束
对外端口	<code>HTTP_PORT</code>	frontend 端口映射	必须选择未被占用的宿主机端口
数据库	数据库名、用户、密码	mysql 与 backend	密码只放本地 <code>deploy.env</code>
跨域	允许来源	后端 CORS/WebSocket	生产环境限定实际访问域名
上传	上传目录	backend 与 <code>upload_data</code> 卷	必须持久化且具备写权限
AI	API 地址、模型、密钥	AI 服务	可为空；不得提交版本库

8.6 模块测试矩阵

模块	正常场景	边界/异常场景	验证点
认证	买家、卖家登录	密码错误、禁用账号、重启后旧 Token	状态码、密码不泄露、重新登录
商品	查询、发布、审核通过	负价格、非卖家发布、拒绝审核	状态和公开可见性
订单	创建、查询、评价	自购、无效商品、重复评价	归属、金额、唯一约束
购物车与地址	添加、改量、勾选、默认地址切换	自购、越权项、无效数量、删除他人地址	服务端持久化、用户归属和默认项一致性
社区	发帖、评论、点赞	空内容、无效话题、重复互动	数据唯一性和列表刷新
聊天	建会话、发信、已读	非参与者访问、断线重连	权限、历史消息、实时推送
部署	一键启动、图片访问	端口占用、数据库未健康、卷重建	健康检查和数据持久化

截至 2026-08-29，后端 JUnit/Mockito 测试已在本机实测 79 项，其中 78 项通过、1 项跳过、0 失败、0 错误。主分支中的 Postman/Newman 集合包含 66 个请求级用例，Selenium 工程包含 14 个场景，二者已完成历史执行；当前待办是补齐 CodeArts 执行号、提交号和原始报告位置。头像上传场景由人工或有头浏览器补测，自动化结果与人工结果分别记录。